

```
1 package com.ejemploSpring.Ejercicio;
2
3 import com.ejemploSpring.Ejercicio.
  dtos.pedido.PedidoCreate;
4 import com.ejemploSpring.Ejercicio.
  dtos.usuario.UsuarioDto;
5 import com.ejemploSpring.Ejercicio.
  entities.Pedido;
6 import com.ejemploSpring.Ejercicio.
  entities.Usuario;
7 import com.ejemploSpring.Ejercicio.
  service.CategoriaService;
8 import com.ejemploSpring.Ejercicio.
  service.PedidoService;
9 import com.ejemploSpring.Ejercicio.
  service.ProductoService;
10 import com.ejemploSpring.Ejercicio.
  service.UsuarioService;
11 import com.ejemploSpring.Ejercicio.
  dtos.categoria.CategoriaCreate;
12 import com.ejemploSpring.Ejercicio.
  dtos.detallePedido.
  DetallePedidoCreate;
13 import com.ejemploSpring.Ejercicio.
  dtos.pedido.PedidoEdit;
14 import com.ejemploSpring.Ejercicio.
  dtos.producto.ProductoCreate;
15 import com.ejemploSpring.Ejercicio.
  dtos.usuario.UsuarioCreate;
```

```
16 import com.ejemploSpring.Ejercicio.  
    enums.Estado;  
17 import com.ejemploSpring.Ejercicio.  
    enums.FormaPago;  
18 import com.ejemploSpring.Ejercicio.  
    enums.Rol;  
19 import org.springframework.boot.  
    CommandLineRunner;  
20 import org.springframework.boot.  
    SpringApplication;  
21 import org.springframework.boot.  
    autoconfigure.SpringBootApplication;  
22 import org.springframework.context.  
    annotation.Bean;  
23 import org.springframework.  
    transaction.annotation.Transactional;  
24  
25 import java.math.BigDecimal;  
26 import java.time.LocalDate;  
27 import java.util.List;  
28  
29 @SpringBootApplication  
30 public class EjercicioApplication {  
31  
32     public static void main(String[]  
        args) {  
33         SpringApplication.run(  
            EjercicioApplication.class, args);  
34     }
```

```

35
36     @Bean
37     public CommandLineRunner initData
38     (
39         UsuarioService
40         usuarioService,
41         CategoriaService
42         categoriaService,
43         ProductoService
44         productoService,
45         PedidoService
46         pedidoService
47     ) {
48         return args -> {
49
50             // =====
51             // 1) USUARIOS (5)
52             // =====
53
54             UsuarioDto u1 =
55             usuarioService.save(new UsuarioCreate
56             (
57
58                 "Matías", "Carro"
59                 , "matias@mail.com", "1155555555", "
60                 123456", Rol.USUARIO
61             ));
62
63             UsuarioDto u2 =

```

```
52 usuarioService.save(new UsuarioCreate
    (
53         "Ana", "Gómez", "
    ana@mail.com", "1144444444", "abc123"
    , Rol.USUARIO
54     ));
55
56     UsuarioDto u3 =
    usuarioService.save(new UsuarioCreate
    (
57         "Luis", "Pérez",
    "luis@mail.com", "1133333333", "
    pass123", Rol.USUARIO
58     ));
59
60     UsuarioDto u4 =
    usuarioService.save(new UsuarioCreate
    (
61         "Carla", "Sosa",
    "carla@mail.com", "1122222222", "
    qwerty", Rol.USUARIO
62     ));
63
64     UsuarioDto u5 =
    usuarioService.save(new UsuarioCreate
    (
65         "Jorge", "Ramírez
    ", "jorge@mail.com", "1111111111", "
    jorgepass", Rol.ADMIN
```

```
66        ));  
67  
68  
69  
70        // =====  
71        // 2) CATEGORÍAS (insumos  
72        de computación)  
73  
74        // =====  
75        var c1 = categoriaService  
76        .save(new CategoriaCreate(  
77            "Periféricos", "  
78            Teclados, mouse, auriculares"  
79            ));  
80  
81        var c2 = categoriaService  
82        .save(new CategoriaCreate(  
83            "Almacenamiento"  
84            , "Discos SSD, HDD, memorias"  
85            ));  
86  
87        var c3 = categoriaService  
88        .save(new CategoriaCreate(  
89            "Componentes", "  
90            Placas de video, motherboards,  
91            procesadores"  
92            ));  
93  
94        var c4 = categoriaService
```

```
84 .save(new CategoriaCreate(
85         "Monitores", "
    Pantallas LED, IPS, 144Hz"
86     ));
87
88     var c5 =
    categoriaService.save(new
    CategoriaCreate(
89         "Accesorios", "
    Cables, adaptadores, soportes"
90     ));
91
92
93
    // =====
94         // 3) PRODUCTOS (10)
95
    // =====
96         productoService.save(new
    ProductoCreate("Teclado Mecánico
    Redragon", new BigDecimal("25000"),
    "Switch Red", 30, "teclado.jpg",
    true, c1.id()));
97         productoService.save(new
    ProductoCreate("Mouse Logitech G203
    ", new BigDecimal("18000"), "RGB
    8000 DPI", 40, "mouse.jpg", true, c1
    .id()));
98         productoService.save(new
```

```
98  ProductoCreate("Auriculares HyperX
    Cloud II", new BigDecimal("45000"),
    "Sonido 7.1", 20, "auriculares.jpg"
    , true, c1.id()));
99
100      productoService.save(new
    ProductoCreate("SSD Kingston 480GB"
    , new BigDecimal("32000"), "SATA 3"
    , 50, "ssd.jpg", true, c2.id()));
101      productoService.save(new
    ProductoCreate("Memoria RAM 16GB
    DDR4", new BigDecimal("28000"), "
    3200MHz", 35, "ram.jpg", true, c2.id
    ());
102
103      productoService.save(new
    ProductoCreate("Motherboard ASUS
    B450", new BigDecimal("65000"), "AM4
    ", 15, "mother.jpg", true, c3.id
    ());
104      productoService.save(new
    ProductoCreate("Placa de Video GTX
    1660", new BigDecimal("180000"), "
    6GB GDDR5", 10, "gtx1660.jpg", true
    , c3.id()));
105
106      productoService.save(new
    ProductoCreate("Monitor Samsung 24
    \", new BigDecimal("90000"), "IPS
```

```

106 75Hz", 25, "monitor.jpg", true, c4.
    id())));
107         productoService.save(new
    ProductoCreate("Cable HDMI 2.0",
    new BigDecimal("5000"), "1.8m", 100
    , "hdmi.jpg", true, c5.id())));
108         productoService.save(new
    ProductoCreate("Soporte para
    Notebook", new BigDecimal("15000"),
    "Ajustable", 60, "soporte.jpg", true
    , c5.id())));
109
110
111
    // =====
112         // 4) PEDIDOS (10)
113
    // =====
114
115         // Pedido 1 - Usuario 1
116         Pedido p1 =
    pedidoService.save(new PedidoCreate(
117             LocalDate.now
    (), Estado.PENDIENTE, FormaPago.
    EFECTIVO,
118             List.of(new
    DetallePedidoCreate(1, 1L), new
    DetallePedidoCreate(1, 4L))
119             ));

```



```
120         pedidoService.  
        asignarPedido(u1.id(), p1);  
121  
122         // Pedido 2 - Usuario 2  
123         Pedido p2 =  
        pedidoService.save(new PedidoCreate(  
124             LocalDate.now  
        ( ), Estado.PENDIENTE, FormaPago.  
        TARJETA,  
125             List.of(new  
        DetallePedidoCreate(2, 2L))  
126             ));  
127         pedidoService.  
        asignarPedido(u2.id(), p2);  
128  
129         // Pedido 3 - Usuario 3  
130         Pedido p3 =  
        pedidoService.save(new PedidoCreate(  
131             LocalDate.now  
        ( ), Estado.PENDIENTE, FormaPago.  
        TRANSFERENCIA,  
132             List.of(new  
        DetallePedidoCreate(1, 3L), new  
        DetallePedidoCreate(1, 5L))  
133             ));  
134         pedidoService.  
        asignarPedido(u3.id(), p3);  
135  
136         // Pedido 4 - Usuario 4
```

```
137         Pedido p4 =
        pedidoService.save(new PedidoCreate(
138             LocalDate.now
            (), Estado.PENDIENTE, FormaPago.
            EFECTIVO,
139             List.of(new
            DetallePedidoCreate(1, 6L))
140             ));
141         pedidoService.
            asignarPedido(u4.id(), p4);
142
143         // Pedido 5 - Usuario 5
144         Pedido p5 =
            pedidoService.save(new PedidoCreate(
145             LocalDate.now
            (), Estado.PENDIENTE, FormaPago.
            TARJETA,
146             List.of(new
            DetallePedidoCreate(1, 7L))
147             ));
148         pedidoService.
            asignarPedido(u5.id(), p5);
149
150         // Pedido 6 - Usuario 1
151         Pedido p6 =
            pedidoService.save(new PedidoCreate(
152             LocalDate.now
            (), Estado.PENDIENTE, FormaPago.
            TRANSFERENCIA,
```

```
153             List.of(new
DetallePedidoCreate(2, 8L))
154         ));
155         pedidoService.
asignarPedido(u1.id(), p6);
156
157         // Pedido 7 - Usuario 2
158         Pedido p7 =
pedidoService.save(new PedidoCreate(
159             LocalDate.now
(), Estado.PENDIENTE, FormaPago.
EFECTIVO,
160             List.of(new
DetallePedidoCreate(3, 9L))
161         ));
162         pedidoService.
asignarPedido(u2.id(), p7);
163
164         // Pedido 8 - Usuario 3
165         Pedido p8 =
pedidoService.save(new PedidoCreate(
166             LocalDate.now
(), Estado.PENDIENTE, FormaPago.
TARJETA,
167             List.of(new
DetallePedidoCreate(1, 10L))
168         ));
169         pedidoService.
asignarPedido(u3.id(), p8);
```

```
170
171         // Pedido 9 - Usuario 4
172         Pedido p9 =
            pedidoService.save(new PedidoCreate(
173             LocalDate.now
            (), Estado.PENDIENTE, FormaPago.
            TRANSFERENCIA,
174             List.of(new
            DetallePedidoCreate(1, 2L), new
            DetallePedidoCreate(1, 5L))
175             ));
176         pedidoService.
            asignarPedido(u4.id(), p9);
177
178         // Pedido 10 - Usuario 5
179         Pedido p10 =
            pedidoService.save(new PedidoCreate(
180             LocalDate.now
            (), Estado.PENDIENTE, FormaPago.
            EFECTIVO,
181             List.of(new
            DetallePedidoCreate(2, 1L), new
            DetallePedidoCreate(1, 7L))
182             ));
183         pedidoService.
            asignarPedido(u5.id(), p10);
184
185
186         System.out.println(">>>
```

```
186 Datos iniciales cargados  
    correctamente.");
```

```
187         };
```

```
188     }
```

```
189
```

```
190
```

```
191 }
```

```
192
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.pedido;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.detallePedido.DetallePedidoDto;  
4 import com.ejemploSpring.Ejercicio.  
  entities.Pedido;  
5 import com.ejemploSpring.Ejercicio.  
  enums.Estado;  
6 import com.ejemploSpring.Ejercicio.  
  enums.FormaPago;  
7  
8 import java.math.BigDecimal;  
9 import java.time.LocalDate;  
10 import java.util.List;  
11  
12 public record PedidoDto(  
13     Long id,  
14     LocalDate fecha,  
15     Estado estado,  
16     BigDecimal total,  
17     FormaPago formapago,  
18     List<DetallePedidoDto>  
    detalles  
19 ) {  
20  
21     public static PedidoDto toDto(  
        Pedido pedido) {  
22         return new PedidoDto(  

```

```
23         pedido.getId(),
24         pedido.getFecha(),
25         pedido.getEstado(),
26         pedido.getTotal(),
27         pedido.getFormapago
28         ( ),
29         pedido.
30         getDetallePedidos().stream()
31         .map(
32         DetallePedidoDto::toDto)
33         .toList()
34     );
35 }
36 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.pedido;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Pedido;  
4 import com.ejemploSpring.Ejercicio.  
  entities.Usuario;  
5 import com.ejemploSpring.Ejercicio.  
  enums.Estado;  
6 import com.ejemploSpring.Ejercicio.  
  enums.FormaPago;  
7  
8 import java.time.LocalDate;  
9  
10 public record PedidoEdit(  
11     LocalDate fecha,  
12     Estado estado,  
13     FormaPago formapago  
14  
15 ) {  
16  
17     public void applyTo(Pedido pedido  
18         , Usuario usuario) {  
19  
20         if (this.fecha != null) {  
21             pedido.setFecha(this.  
22                 fecha);  
23         }  
24     }  
25 }
```



```
23         if (this.estado != null) {
24             pedido.setEstado(this.
                estado);
25         }
26
27         if (this.formapago != null) {
28             pedido.setFormapago(this.
                formapago);
29         }
30
31         /*         if (this.usuarioId != null
                && usuario != null) {
32             pedido.setUsuario(usuario
                );
33         }*/
34     }
35
36
37 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.pedido;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.detallePedido.  
  DetallePedidoCreate;  
4 import com.ejemploSpring.Ejercicio.  
  enums.Estado;  
5 import com.ejemploSpring.Ejercicio.  
  enums.FormaPago;  
6  
7 import java.time.LocalDate;  
8 import java.util.List;  
9  
10 public record PedidoCreate(  
11     LocalDate fecha,  
12     Estado estado,  
13     FormaPago formapago,  
14     List<DetallePedidoCreate>  
    detalles  
15 ) {}  
16
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.pedido;  
2  
3 public record PedidoCreateConUsuario(  
4     Long usuarioId,  
5     PedidoCreate pedido  
6 ) {}  
7
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.usuario;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Usuario;  
4 import com.ejemploSpring.Ejercicio.  
  enums.Rol;  
5  
6 public record UsuarioDto(  
7     Long id,  
8     String nombre,  
9     String apellido,  
10    String mail,  
11    String celular,  
12    Rol rol  
13 ) {  
14  
15     public static UsuarioDto toDto(  
        Usuario usuario) {  
16         return new UsuarioDto(  
17             usuario.getId(),  
18             usuario.getNombre(),  
19             usuario.getApellido  
        (),  
20             usuario.getMail(),  
21             usuario.getCelular(),  
22             usuario.getRol()  
23         );  
24     }
```

25 }

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.usuario;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Usuario;  
4 import com.ejemploSpring.Ejercicio.  
  enums.Rol;  
5  
6 import java.util.Optional;  
7  
8 public record UsuarioEdit(  
9     String nombre,  
10    String apellido,  
11    String mail,  
12    String celular,  
13    String contrasenia,  
14    Rol rol  
15 ) {  
16  
17     public void applyTo(Usuario  
18     usuario) {  
19         if (this.nombre != null) {  
20             usuario.setNombre(this.  
21             nombre);  
22         }  
23         if (this.apellido != null) {  
24             usuario.setApellido(this.
```

```
24 apellido);
25     }
26
27     if (this.mail != null) {
28         usuario.setMail(this.mail
29     );
30     }
31
32     if (this.celular != null) {
33         usuario.setCelular(this.
34     celular);
35     }
36
37     if (this.contrasenia != null
38     ) {
39         usuario.setContrasenia(
40     this.contrasenia);
41     }
42 }
43 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.usuario;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Usuario;  
4 import com.ejemploSpring.Ejercicio.  
  enums.Rol;  
5  
6 public record UsuarioCreate(  
7     String nombre,  
8     String apellido,  
9     String mail,  
10    String celular,  
11    String contrasenia,  
12    Rol rol  
13 ) {  
14     public Usuario toEntity(Usuario  
  usuario){  
15         return Usuario.builder()  
16             .nombre(this.nombre)  
17             .apellido(this.  
  apellido)  
18             .mail(this.mail)  
19             .celular(this.celular  
  )  
20             .contrasenia(this.  
  contrasenia)  
21             .rol(this.rol)  
22             .build();
```


23

24 }

25

26 }

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.usuario;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.usuario.UsuarioDto;  
4 import com.ejemploSpring.Ejercicio.  
  entities.Usuario;  
5 import com.ejemploSpring.Ejercicio.  
  enums.Rol;  
6  
7 public record UsuarioNombreDto(  
8     String nombre,  
9     String apellido  
10 ) {  
11  
12     public static UsuarioNombreDto  
toDto(Usuario usuario) {  
13         return new UsuarioNombreDto(  
14             usuario.getNombre(),  
15             usuario.getApellido()  
16         );  
17     }  
18 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.producto;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaDto;  
4 import com.ejemploSpring.Ejercicio.  
  entities.Categoria;  
5 import com.ejemploSpring.Ejercicio.  
  entities.Producto;  
6  
7 import java.math.BigDecimal;  
8  
9 public record ProductoDto(  
10     Long id,  
11     String nombre,  
12     BigDecimal precio,  
13     String descripcion,  
14     int stock,  
15     String imagen,  
16     Boolean disponible  
17     /*      CategoriaDto categoriaDto*/  
18 ){  
19     public static ProductoDto toDto(  
    Producto producto){  
20         return new ProductoDto(  
21             producto.getId(),  
22             producto.getNombre(),  
23             producto.getPrecio(),  
24             producto.
```

```
24 getDescripcion(),
25         producto.getStock(),
26         producto.getImagen(),
27         producto.isDisponible
    ());
28         // Si producto.
    getCategoria es nulo, se devuelve
    nulo
29 /*         producto.
    getCategoria() != null ? CategoriaDto
    .toDto(producto.getCategoria()) :
    null*/
30
31     }
32 }
33
34
35
```

```
1 package com.ejemploSpring.Ejercicio.  
   dtos.producto;  
2  
3 import com.ejemploSpring.Ejercicio.  
   entities.Categoria;  
4 import com.ejemploSpring.Ejercicio.  
   entities.Producto;  
5  
6 import java.math.BigDecimal;  
7  
8 public record ProductoEdit(  
9     String nombre,  
10    BigDecimal precio,  
11    String descripcion,  
12    Integer stock,  
13    String imagen,  
14    Boolean disponible,  
15    Long idCategoria  
16 ) {  
17     public void applyTo(Producto  
        producto, Categoria categoria){  
18         if (this.nombre != null) {  
19             producto.setNombre(this.  
                nombre);  
20         }  
21         if (this.precio != null) {  
22             producto.setPrecio(this.  
                precio);  
23         }  
}
```

```
24         if (this.descripcion != null
25         ) {
26             producto.setDescripcion(
27             this.descripcion);
28         }
29         if (this.stock != null) {
30             producto.setStock(this.
31             stock);
32         }
33         if (this.imagen != null) {
34             producto.setImagen(this.
35             imagen);
36         }
37         if (this.disponible != null
38         ) {
39             producto.setDisponible(
40             this.disponible);
41         }
42     }
43     /*
44         if (categoria != null) {
45             producto.setCategoria(
46             categoria);
47         }
48     */
49 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.producto;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Categoria;  
4 import com.ejemploSpring.Ejercicio.  
  entities.Producto;  
5  
6 import java.math.BigDecimal;  
7  
8 public record ProductoCreate(  
9     String nombre,  
10    BigDecimal precio,  
11    String descripcion,  
12    int stock,  
13    String imagen,  
14    Boolean disponible,  
15    Long idCategoria  
16 ) {  
17     public Producto toEntity(  
        Categoria categoria){  
18         return Producto.builder()  
19             .nombre(this.nombre)  
20             .precio(this.precio)  
21             .descripcion(this.  
        descripcion)  
22             .stock(this.stock)  
23             .imagen(this.imagen)  
24             .disponible(this.
```

```
24 disponible != null ? this.disponible
    : false)
25             /*.categoria(
    categoria)*/
26             .build();
27     }
28 }
29
```



```
1 package com.ejemploSpring.Ejercicio.  
  dtos.categoria;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Categoria;  
4  
5 public record CategoriaDto(  
6     Long id,  
7     String nombre,  
8     String descripcion,  
9     Boolean eliminado  
10 ) {  
11     public static CategoriaDto toDto(  
        Categoria categoria){  
12         return new CategoriaDto(  
13             categoria.getId(),  
14             categoria.getNombre  
15             (),  
16             categoria.  
        getDescripcion(),  
17             categoria.  
        getEliminado()  
18         );  
19     }  
20 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.categoria;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Categoria;  
4  
5 public record CategoriaEdit(  
6     String nombre,  
7     String descripcion  
8 ) {  
9     public void applyTo(Categoria  
categoria){  
10         if(this.nombre != null){  
11             categoria.setNombre(this.  
nombre);  
12         }  
13  
14         if(this.descripcion != null){  
15             categoria.setDescripcion(  
this.descripcion);  
16         }  
17     }  
18 }  
19
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.categoria;  
2 import com.ejemploSpring.Ejercicio.  
  entities.Categoria;  
3  
4 public record CategoriaCreate(  
5     String nombre,  
6     String descripcion) {  
7  
8  
9     public Categoria toEntity(){  
10 //         return new Categoria(this.  
    nombre,this.descripcion);  
11         return Categoria.builder  
    ()  
12             .nombre(this.  
    nombre)  
13             .descripcion(this  
    .descripcion)  
14             .build();  
15     }  
16 }  
17  
18
```

```
1 package com.ejemploSpring.Ejercicio.  
   dtos.detallePedido;  
2  
3 import com.ejemploSpring.Ejercicio.  
   entities.DetallePedido;  
4  
5 import java.math.BigDecimal;  
6  
7 public record DetallePedidoDto(  
8     Long id,  
9     int cantidad,  
10    BigDecimal subtotal,  
11    Long productoId,  
12    String productoNombre  
13    /*      Long pedidoId*/  
14 ) {  
15  
16  
17     public static DetallePedidoDto  
toDto(DetallePedido detalle) {  
18         return new DetallePedidoDto(  
19             detalle.getId(),  
20             detalle.getCantidad  
21             (),  
22             detalle.getSubtotal  
23             (),  
24             detalle.getProducto  
25             ().getId(),  
26             detalle.getProducto
```

```
23 ().getNombre());  
24 /*                               detalle.getPedido  
    ().getId()*/  
25  
26     }  
27 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  dtos.detallePedido;  
2  
3 public record DetallePedidoCreate(  
4     int cantidad,  
5     Long productoId  
6 ){}  
7  
8  
9
```

```
1 package com.ejemploSpring.Ejercicio.  
  enums;  
2  
3 public enum Rol {  
4     ADMIN, USUARIO  
5 }  
6
```

```
1 package com.ejemploSpring.Ejercicio.  
  enums;  
2  
3 public enum Estado {  
4     PENDIENTE, CONFIRMADO, TERMINADO  
  , CANCELADO  
5 }  
6
```



```
1 package com.ejemploSpring.Ejercicio.  
  enums;  
2  
3 public enum FormaPago {  
4     TARJETA, TRANSFERENCIA, EFECTIVO  
5 }  
6
```

```
1 package com.ejemploSpring.Ejercicio.  
  service;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.pedido.PedidoCreate;  
4 import com.ejemploSpring.Ejercicio.  
  dtos.pedido.PedidoEdit;  
5 import com.ejemploSpring.Ejercicio.  
  entities.Pedido;  
6  
7 import java.util.List;  
8  
9 public interface PedidoService {  
10  
11     Pedido save(PedidoCreate  
  pedidoCreate);  
12  
13     Pedido findById(Long id);  
14  
15     List<Pedido> findAll();  
16  
17     Pedido update(Long id, PedidoEdit  
  pedidoEdit);  
18  
19     void asignarPedido(Long usuarioId  
  , Pedido pedido);  
20  
21     void deleteById(Long id);  
22
```

23 }

```
1 package com.ejemploSpring.Ejercicio.  
  service;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.usuario.UsuarioCreate;  
4 import com.ejemploSpring.Ejercicio.  
  dtos.usuario.UsuarioDto;  
5 import com.ejemploSpring.Ejercicio.  
  dtos.usuario.UsuarioEdit;  
6 import com.ejemploSpring.Ejercicio.  
  dtos.usuario.UsuarioNombreDto;  
7 import com.ejemploSpring.Ejercicio.  
  entities.Pedido;  
8 import com.ejemploSpring.Ejercicio.  
  entities.Usuario;  
9  
10 import java.util.List;  
11  
12 public interface UsuarioService {  
13     public UsuarioDto save(  
        UsuarioCreate usuarioCreate);  
14  
15     public UsuarioDto findById(Long  
        id);  
16  
17     public List<UsuarioDto> findAll  
        ();  
18  
19     public UsuarioDto update(
```

```
19 UsuarioEdit usuarioEdit, Long
    idUsuario);
20
21     public void deleteById(Long id);
22
23     public UsuarioDto findByMail(
    String email);
24
25     public UsuarioNombreDto
    getNombreApellido(Long id);
26
27 }
28
```

```
1 package com.ejemploSpring.Ejercicio.  
  service;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.producto.ProductoCreate;  
4 import com.ejemploSpring.Ejercicio.  
  dtos.producto.ProductoDto;  
5 import com.ejemploSpring.Ejercicio.  
  dtos.producto.ProductoEdit;  
6 import com.ejemploSpring.Ejercicio.  
  entities.Producto;  
7  
8 import java.util.List;  
9  
10 public interface ProductoService {  
11     public ProductoDto save(  
        ProductoCreate productoCreate);  
12     public ProductoDto findById(Long  
        id);  
13     public List<ProductoDto> findAll  
        ();  
14     public ProductoDto update(  
        ProductoEdit productoEdit, Long  
        idProducto);  
15     public Producto findEntityById(  
        Long id);  
16     public void deleteById(Long id);  
17     public List<ProductoDto>  
        findAllEliminados();
```

18 }

19

```
1 package com.ejemploSpring.Ejercicio.  
  service;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaCreate;  
4 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaDto;  
5 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaEdit;  
6 import com.ejemploSpring.Ejercicio.  
  entities.Categoria;  
7  
8 import java.util.List;  
9  
10 public interface CategoriaService {  
11     public CategoriaDto save(  
        CategoriaCreate categoriaCreate);  
12     public CategoriaDto findById(Long  
        id);  
13     public List<CategoriaDto> findAll  
        ();  
14     public CategoriaDto update(  
        CategoriaEdit categoriaEdit, Long  
        idCategoria);  
15     public void deleteById(Long id);  
16  
17 }  
18
```



```
1 package com.ejemploSpring.Ejercicio.  
    service;  
2  
3 import com.ejemploSpring.Ejercicio.  
    dtos.detallePedido.  
    DetallePedidoCreate;  
4 import com.ejemploSpring.Ejercicio.  
    dtos.pedido.PedidoCreate;  
5 import com.ejemploSpring.Ejercicio.  
    dtos.pedido.PedidoEdit;  
6 import com.ejemploSpring.Ejercicio.  
    entities.Pedido;  
7 import com.ejemploSpring.Ejercicio.  
    entities.Producto;  
8 import com.ejemploSpring.Ejercicio.  
    entities.Usuario;  
9 import com.ejemploSpring.Ejercicio.  
    repository.DetallePedidoRepository;  
10 import com.ejemploSpring.Ejercicio.  
    repository.PedidoRepository;  
11 import com.ejemploSpring.Ejercicio.  
    repository.ProductoRepository;  
12 import com.ejemploSpring.Ejercicio.  
    repository.UsuarioRepository;  
13 import lombok.RequiredArgsConstructor  
    ;  
14 import org.springframework.stereotype  
    .Service;  
15 import org.springframework.
```

```
15 transaction.annotation.Transactional;
16
17 import java.util.List;
18
19 @Service
20 @RequiredArgsConstructor
21 public class PedidoServiceImpl
22     implements PedidoService {
23
24     private final PedidoRepository
25         pedidoRepository;
26     private final ProductoService
27         productoService;
28     private final UsuarioRepository
29         usuarioRepository;
30     private final ProductoRepository
31         productoRepository;
32     private final
33         DetallePedidoRepository
34         detallePedidoRepository ;
35
36     @Override
37     public Pedido save(PedidoCreate
38         pedidoCreate) {
39
40         Pedido pedido = Pedido.
41             builder()
42                 .fecha(pedidoCreate.
43                     fecha())
```

```
34         .estado(pedidoCreate.  
    estado())  
35         .formapago(  
    pedidoCreate.formapago())  
36         .build();  
37  
38     for (DetallePedidoCreate d :  
    pedidoCreate.detalles()) {  
39         Producto producto =  
    productoService.findEntityById(d.  
    productoId());  
40         pedido.addDetallePedido(d  
    .cantidad(), producto);  
41     }  
42  
43     pedido.calcularTotal();  
44  
45     return pedidoRepository.save(  
    pedido);  
46 }  
47  
48 @Override  
49 public Pedido findById(Long id) {  
50     return pedidoRepository.  
    findById(id)  
51     .orElseThrow(() ->  
    new RuntimeException("Pedido no  
    encontrado"));  
52 }
```

```
53
54     @Override
55     public List<Pedido> findAll() {
56         return pedidoRepository.
findAll();
57     }
58
59     @Override
60     public Pedido update(Long id,
PedidoEdit pedidoEdit) {
61         Pedido pedido = findById(id);
62
63         pedido.setFecha(pedidoEdit.
fecha());
64         pedido.setEstado(pedidoEdit.
estado());
65         pedido.setFormapago(
pedidoEdit.formapago());
66
67         pedido.calcularTotal();
68
69         return pedidoRepository.save(
pedido);
70     }
71
72     @Override
73     public void deleteById(Long id
) { //Baja logica
74         Pedido pedido =
```

```
74 pedidoRepository.findById(id)
75         .orElseThrow(() ->
    new NullPointerException("No se
    encontró pedido con id " + id));
76
77         pedido.setEliminado(true);
78         pedidoRepository.save(pedido
    );
79     }
80
81
82     @Transactional
83     public void asignarPedido(Long
    usuarioId, Pedido pedido) {
84         Usuario usuario =
    usuarioRepository.findById(usuarioId
    )
85         .orElseThrow(() ->
    new RuntimeException("Usuario no
    encontrado"));
86
87         usuario.addPedido(pedido);
88     }
89
90 }
```

```
1 package com.ejemploSpring.Ejercicio.  
    service;  
2  
3 import com.ejemploSpring.Ejercicio.  
    dtos.usuario.UsuarioCreate;  
4 import com.ejemploSpring.Ejercicio.  
    dtos.usuario.UsuarioDto;  
5 import com.ejemploSpring.Ejercicio.  
    dtos.usuario.UsuarioEdit;  
6 import com.ejemploSpring.Ejercicio.  
    dtos.usuario.UsuarioNombreDto;  
7 import com.ejemploSpring.Ejercicio.  
    entities.Pedido;  
8 import com.ejemploSpring.Ejercicio.  
    entities.Usuario;  
9 import com.ejemploSpring.Ejercicio.  
    repository.UsuarioRepository;  
10 import lombok.RequiredArgsConstructor  
    ;  
11 import org.springframework.stereotype  
    .Service;  
12 import org.springframework.  
    transaction.annotation.Transactional;  
13  
14 import java.util.List;  
15  
16 @Service  
17 @RequiredArgsConstructor  
18 public class UsuarioServiceImpl
```

```
18 implements UsuarioService {
19
20     private final UsuarioRepository
        usuarioRepository;
21
22     @Override
23     public UsuarioDto save(
        UsuarioCreate usuarioCreate) {
24
25         Usuario usuario =
        usuarioCreate.toEntity(new Usuario
            ());
26
27         Usuario guardado =
        usuarioRepository.save(usuario);
28
29         return UsuarioDto.toDto(
        guardado);
30     }
31
32     @Override
33     public UsuarioDto findById(Long
        id) {
34
35         Usuario usuario =
        usuarioRepository.findById(id)
36             .orElseThrow(() ->
        new RuntimeException("Usuario no
        encontrado"));
}
```

```
37
38         return UsuarioDto.toDto(
39             usuario);
40
41     @Override
42     public List<UsuarioDto> findAll
43     () {
44
45         return usuarioRepository.
46             findAll()
47             .stream()
48             .map(UsuarioDto::
49                 toDto)
50             .toList();
51     }
52
53     @Override
54     public UsuarioDto update(
55         UsuarioEdit usuarioEdit, Long
56         idUsuario) {
57
58         Usuario usuario =
59             usuarioRepository.findById(idUsuario)
60             .orElseThrow(() ->
61                 new RuntimeException("Usuario no
62                 encontrado"));
63
64         usuarioEdit.applyTo(usuario);
```



```
57
58         Usuario actualizado =
        usuarioRepository.save(usuario);
59
60         return UsuarioDto.toDto(
        actualizado);
61     }
62
63     @Override
64     public void deleteById(Long id) {
65
66         if (!usuarioRepository.
        existsById(id)) {
67             throw new
        RuntimeException("Usuario no
        encontrado");
68         }
69
70         usuarioRepository.deleteById(
        id);
71     }
72
73     @Override
74     public UsuarioDto findByMail(
        String mail) {
75         Usuario usuario =
        usuarioRepository.findByMail(mail)
76             .orElseThrow(() ->
        new NullPointerException("No se
```

```
76 encontró usuario con email " + mail
   ));
77
78         return UsuarioDto.toDto(
   usuario);
79     }
80
81     @Override
82     public UsuarioNombreDto
   getNombreApellido(Long id) {
83
84         Usuario usuario =
   usuarioRepository.findById(id)
85             .orElseThrow(() ->
   new RuntimeException("Usuario no
   encontrado"));
86
87         return UsuarioNombreDto.
   toDto(usuario);
88     }
89
90
91 }
```

```
1 package com.ejemploSpring.Ejercicio.  
    service;  
2  
3 import com.ejemploSpring.Ejercicio.  
    dtos.producto.ProductoCreate;  
4 import com.ejemploSpring.Ejercicio.  
    dtos.producto.ProductoDto;  
5 import com.ejemploSpring.Ejercicio.  
    dtos.producto.ProductoEdit;  
6 import com.ejemploSpring.Ejercicio.  
    entities.Categoria;  
7 import com.ejemploSpring.Ejercicio.  
    entities.Producto;  
8 import com.ejemploSpring.Ejercicio.  
    repository.CategoriaRepository;  
9 import com.ejemploSpring.Ejercicio.  
    repository.ProductoRepository;  
10 import lombok.RequiredArgsConstructor  
    ;  
11 import org.springframework.stereotype  
    .Service;  
12  
13 import java.util.List;  
14  
15 @Service  
16 @RequiredArgsConstructor //reemplaza  
    el productoserviceimpl  
17 public class ProductoServiceImpl  
    implements ProductoService {
```

```
18
19     private final ProductoRepository
    productoRepository;
20     private final CategoriaRepository
    categoriaRepository;
21
22     /*      public ProductServiceImpl(
    ProductoRepository productoRepository
    , CategoriaRepository
    categoriaRepository) {
23         this.productoRepository =
    productoRepository;
24         this.categoriaRepository =
    categoriaRepository;
25     */
26
27     @Override
28     public ProductoDto save(
    ProductoCreate productoCreate) {
29         Categoria categoria =
    categoriaRepository.findById(
    productoCreate.idCategoria()).
    orElseThrow(() -> new
    NullPointerException("No se encontro
categoria con id" + productoCreate.
    idCategoria()));
30         Producto producto =
    productoCreate.toEntity(categoria);
31         producto = productoRepository
```

```
31 .save(producto);
32     return ProductoDto.toDto(
    producto);
33 }
34
35     @Override
36     public ProductoDto findById(Long
    id) {
37         Producto producto =
    productoRepository.findById(id).
    orElseThrow(() -> new
    NullPointerException("No se encontro
    categoria con id" + id));
38         return ProductoDto.toDto(
    producto);
39     }
40
41     //Buscar todos los disponibles (
eliminado = false)
42     @Override
43     public List<ProductoDto> findAll
    () {
44         return productoRepository.
    findAllByEliminadoFalse()
45             .stream()
46             .map(ProductoDto::
    toDto)
47             .toList();
48     }
```

```
49
50     //encontrar todos los eliminados
51     @Override
52     public List<ProductoDto>
        findAllEliminados() {
53         return productoRepository.
            findAllByEliminadoTrue()
54             .stream()
55             .map(ProductoDto::
                toDto)
56             .toList();
57     }
58
59     @Override
60     public ProductoDto update(
        ProductoEdit productoEdit, Long
        idProducto) {
61         Producto producto =
            productoRepository.findById(
                idProducto).orElseThrow(() -> new
                NullPointerException("No se encontro
                categoria con id" + idProducto));
62         Categoria categoria = null;
63         if(productoEdit.idCategoria
            () != null) {
64             categoria =
                categoriaRepository.findById(
                    productoEdit.idCategoria()).
                    orElseThrow(() -> new
```

```
64 NullPointerException(("No se encontro
    categoria con id" + idProducto)));
65     }
66     productoEdit.applyTo(producto
    , categoria);
67     producto = productoRepository
    .save(producto);
68     return ProductoDto.toDto(
    producto);
69     }
70
71     //Eliminado logico
72     @Override
73     public void deleteById(Long id) {
74         Producto producto =
    productoRepository.findById(id).
    orElseThrow(() -> new
    NullPointerException(("No se encontro
    categoria con id" + id)));
75         producto.setEliminado(true);
76         productoRepository.save(
    producto);
77
78     }
79
80     @Override
81     public Producto findEntityById(
    Long id) {
82         return productoRepository.
```

```
82 findById(id)
83             .orElseThrow(() ->
    new NullPointerException("No se
    encontró producto con id " + id));
84     }
85 }
86
```



```
1 package com.ejemploSpring.Ejercicio.  
  service;  
2  
3 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaCreate;  
4 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaDto;  
5 import com.ejemploSpring.Ejercicio.  
  dtos.categoria.CategoriaEdit;  
6 import com.ejemploSpring.Ejercicio.  
  entities.Categoria;  
7 import com.ejemploSpring.Ejercicio.  
  repository.CategoriaRepository;  
8 import org.springframework.stereotype  
  .Service;  
9  
10 import java.util.List;  
11  
12 @Service  
13 public class CategoriaServiceImpl  
  implements CategoriaService{  
14     private final CategoriaRepository  
        categoriaRepository;  
15  
16     public CategoriaServiceImpl(  
        CategoriaRepository  
        categoriaRepository) {  
17         this.categoriaRepository =  
            categoriaRepository;  
18     }  
19 }
```

```
18     }
19
20     @Override
21     public CategoriaDto save(
22         CategoriaCreate categoriaCreate) {
23         Categoria categoria =
24             categoriaCreate.toEntity();
25         categoria =
26             categoriaRepository.save(categoria);
27         return CategoriaDto.toDto(
28             categoria);
29     }
30
31     @Override
32     public CategoriaDto findById(Long
33         id) {
34         Categoria categoria =
35             categoriaRepository.findById(id)
36                 .filter(c -> !Boolean
37                     .TRUE.equals(c.getEliminado())) //
38                     descarta eliminadas
39                 .orElseThrow(() ->
40                     new NullPointerException("No se
41                         encontró categoría activa con el id "
42                             + id));
43         return CategoriaDto.toDto(
44             categoria);
45     }
46
47 }
```

```
35     @Override
36     public List<CategoriaDto> findAll
37     () {
38         return categoriaRepository.
39         findAllByEliminadoFalse()
40         .stream()
41         .map(CategoriaDto::
42         toDto)
43         .toList();
44     }
45
46     @Override
47     public CategoriaDto update(
48     CategoriaEdit categoriaEdit, Long
49     idCategoria) {
50         Categoria categoria =
51         categoriaRepository.findById(
52         idCategoria).orElseThrow(() -> new
53         NullPointerException("No se contro
54         categoria con el id" + idCategoria));
55         categoriaEdit.applyTo(
56         categoria);
57         categoria =
58         categoriaRepository.save(categoria);
59         return CategoriaDto.toDto(
60         categoria);
61     }
62
63     @Override
```

```
52     public void deleteById(Long id) {
53         Categoria categoria =
categoriaRepository.findById(id)
54             .orElseThrow(() ->
new NullPointerException("No se
contro categoria con el id " + id));
55         categoria.setEliminado(true);
56         categoriaRepository.save(
categoria);
57     }
58 }
59
```

```
1 package com.ejemploSpring.Ejercicio.  
   entities;  
2 import jakarta.persistence.  
   GeneratedValue;  
3 import jakarta.persistence.  
   GenerationType;  
4 import jakarta.persistence.Id;  
5 import jakarta.persistence.  
   MappedSuperclass;  
6 import lombok.*;  
7 import lombok.experimental.  
   SuperBuilder;  
8 import java.time.LocalDateTime;  
9  
10 @MappedSuperclass  
11 @Getter  
12 @Setter  
13 @ToString  
14 @SuperBuilder  
15 @AllArgsConstructor  
16 @NoArgsConstructor  
17 @EqualsAndHashCode(of = {"id"})  
18 public abstract class Base {  
19  
20     //      protected static long idCounter  
        = 0;  
21  
22  
23     @Id
```

```
24     @GeneratedValue(strategy =
      GenerationType.IDENTITY)
25     private Long id;
26     @Builder.Default
27     protected Boolean eliminado =
      false;
28     @Builder.Default
29     protected LocalDateTime createdAt
      = LocalDateTime.now();
30 }
```

```
1 package com.ejemploSpring.Ejercicio.  
   entities;  
2  
3 import com.ejemploSpring.Ejercicio.  
   enums.Estado;  
4 import com.ejemploSpring.Ejercicio.  
   enums.FormaPago;  
5 import jakarta.persistence.*;  
6 import lombok.*;  
7 import lombok.experimental.  
   SuperBuilder;  
8  
9 import java.math.BigDecimal;  
10 import java.time.LocalDate;  
11 import java.util.HashSet;  
12 import java.util.Set;  
13  
14 @Entity  
15 @Table(name = "pedidos")  
16 @Getter  
17 @Setter  
18 @ToString  
19 @EqualsAndHashCode(callSuper = true)  
20 @AllArgsConstructor  
21 @NoArgsConstructor  
22 @SuperBuilder  
23 public class Pedido extends Base  
   implements Calculable {  
24     private LocalDate fecha;
```

```
25     @Enumerated(EnumType.STRING)
26     private Estado estado;
27     private BigDecimal total;
28     @Enumerated(EnumType.STRING)
29     private FormaPago formapago;
30     //se elimina por correccion
31     /*      @ManyToOne
32             @JoinColumn(name = "usuario_id")
33             private Usuario usuario;*/
34
35
36     //se elimina para corregir
direccion de relacion
37     /*      @Builder.Default
38             @OneToMany(mappedBy = "pedido",
39                     cascade = CascadeType.ALL,
40                     orphanRemoval = true)*/
39     @Builder.Default
40     @OneToMany(cascade = CascadeType.
41             ALL, orphanRemoval = true)
42     @JoinColumn(name = "pedido_id")
43     // FK en detallePedido
44     private Set<DetallePedido>
45     detallePedidos = new HashSet<>();
46
47
48     //Agregar detalle
49     public void addDetallePedido(int
50     cantidad, Producto producto) {
```



```
47
48         boolean exists =
detallePedidos.stream()
49             .anyMatch(
detallePedido -> detallePedido.
getProducto().equals(producto));
50         if (exists) return;
51
52         DetallePedido detallePedido
= DetallePedido.builder()
53             .cantidad(cantidad)
54             .producto(producto)
55             .subtotal(producto.
getPrecio().multiply(BigDecimal.
valueOf(cantidad)))
56             //.pedido(this)
57             //se elimina pedido
por correccion
58             .build();
59
60         detallePedidos.add(
detallePedido);
61
62     }
63
64
65     @Override
66     public void calcularTotal() {
67         BigDecimal totalFinal =
```

```
67 detallePedidos.stream()
68     .map(DetallePedido::
    getSubtotal)
69     .reduce(BigDecimal.
    ZERO, BigDecimal::add);
70     this.total = totalFinal;
71 }
72
73
74
75 }
```

```
1 package com.ejemploSpring.Ejercicio.  
   entities;  
2  
3 import com.ejemploSpring.Ejercicio.  
   enums.Rol;  
4 import jakarta.persistence.*;  
5 import lombok.*;  
6 import lombok.experimental.  
   SuperBuilder;  
7  
8 import java.util.HashSet;  
9 import java.util.Set;  
10  
11 @Entity  
12 @Table(name = "usuarios")  
13 @Getter  
14 @Setter  
15  
16 @ToString(callSuper = true, exclude  
   = "contrasenia")  
17 @EqualsAndHashCode(callSuper = false  
   , of = {"mail"})  
18  
19 @SuperBuilder  
20 @NoArgsConstructor  
21 @AllArgsConstructor  
22 public class Usuario extends Base {  
23     private String nombre;  
24     private String apellido;
```

```
25     @Column(unique = true)
26     private String mail;
27     private String celular;
28     private String contrasenia;
29     @Enumerated(EnumType.STRING)
30     private Rol rol;
31
32     //sets de pedidos
33
34
35     @Builder.Default
36     //se elimina por correccion
37     /*      @OneToMany(mappedBy = "usuario
38              *)*/
39     @OneToMany
40     @JoinColumn(name = "usuario_id")
41     // FK en pedido
42     private Set<Pedido> pedidos = new
43     HashSet<>();
44     public void addPedido(Pedido
45     pedido) {
46         pedidos.add(pedido);
47     }
```

```
1 package com.ejemploSpring.Ejercicio.  
  entities;  
2  
3 import jakarta.persistence.*;  
4 import lombok.*;  
5 import lombok.experimental.  
  SuperBuilder;  
6  
7 import java.math.BigDecimal;  
8  
9 @Entity  
10 @Table(name = "productos")  
11 @Getter  
12 @Setter  
13 @ToString  
14 @EqualsAndHashCode(callSuper = false  
    , of = {"nombre", "precio"})  
15 @SuperBuilder  
16 @NoArgsConstructor  
17 @AllArgsConstructor  
18 public class Producto extends Base {  
19     @Column(unique = true)  
20     private String nombre;  
21     private BigDecimal precio;  
22     private String descripcion;  
23     private int stock;  
24     private String imagen;  
25     private boolean disponible;  
26     //Eliminado para cumplir las
```

```
26 relaciones
27 /*      @ManyToOne
28      @JoinColumn(name = "categoria_id
      ")
29      private Categoria categoria;
30
31      //Redundante
32      @Column(name = "ELIMINADO")
33      @Builder.Default
34      private Boolean eliminado = false
      ; //valor por defecto
35
36 */
37
38 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  entities;  
2  
3 import jakarta.persistence.*;  
4 import lombok.*;  
5 import lombok.experimental.  
  SuperBuilder;  
6  
7 import java.util.HashSet;  
8 import java.util.Set;  
9  
10 @Entity  
11 @Table(name = "categorias")  
12 @SuperBuilder  
13 @Getter  
14 @Setter  
15 @ToString(callSuper = true, exclude  
    = "productos")  
16 @EqualsAndHashCode(callSuper = false  
    , of = {"nombre"})  
17 @AllArgsConstructor  
18 @NoArgsConstructor  
19 public class Categoria extends Base {  
20     @Column(unique = true)  
21     private String nombre;  
22     private String descripcion;  
23     //Eliminado para cumplir las  
    relaciones  
24 /*     @OneToMany(mappedBy = "
```

```
24 categoria", cascade = CascadeType.ALL
    , orphanRemoval = true)*/
25 @OneToMany
26 @JoinColumn(name = "categoria_id")
    // FK en producto
27     @Builder.Default
28     private Set<Producto> productos
        = new HashSet<>();
29 /*
30     @Builder.Default
31     private Boolean eliminado = false
        ; //valor por defecto
32 */
33
34
35     //helper para mantener la
        bidireccionalidad - eliminado para
        cumplir la relacion
36 /*     public void addProducto(
        Producto producto) {
37         productos.add(producto);
38         producto.setCategoria(this);
39     }*/
40 }
41
42
43
44
```



```
1 package com.ejemploSpring.Ejercicio.  
   entities;  
2  
3 public interface Calculable{  
4     void calcularTotal();  
5 }  
6  
7
```

```
1 package com.ejemploSpring.Ejercicio.  
   entities;  
2  
3 import jakarta.persistence.Entity;  
4 import jakarta.persistence.JoinColumn  
   ;  
5 import jakarta.persistence.ManyToOne;  
6 import jakarta.persistence.Table;  
7 import lombok.*;  
8 import lombok.experimental.  
   SuperBuilder;  
9  
10 import java.math.BigDecimal;  
11  
12 @Entity  
13 @Table(name = "detalle_pedidos")  
14 @Getter  
15 @Setter  
16 @EqualsAndHashCode(callSuper = false  
   , of = {"producto"})  
17 @NoArgsConstructor  
18 @AllArgsConstructor  
19 @SuperBuilder  
20 public class DetallePedido extends  
   Base {  
21  
22     private int cantidad;  
23     private BigDecimal subtotal;  
24
```

```
25     @ManyToOne
26     @JoinColumn(name = "producto_id")
27     private Producto producto;
28     //Se elimina para cumplir con la
    direccionalidad
29 /*     @ManyToOne
30     @JoinColumn(name = "pedido_id",
    nullable = false)
31     private Pedido pedido;*/
32
33     public DetallePedido(int cantidad
    , BigDecimal subtotal) {
34         this.cantidad = cantidad;
35         this.subtotal = subtotal;
36
37     }
38 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  controller;  
2  
3 import org.springframework.web.bind.  
  annotation.GetMapping;  
4 import org.springframework.web.bind.  
  annotation.RestController;  
5  
6 @RestController  
7 public class HomeController {  
8  
9     @GetMapping("/")  
10    public String home() {  
11        return ""  
12  
13        <style>  
14            body { font-  
family: Arial; padding: 20px; }  
15            h1 { margin-  
bottom: 10px; }  
16            pre { background  
: #f4f4f4; padding: 15px; border-  
radius: 5px; }  
17            a { color: blue  
; }  
18  
19            </style>  
20  
21            <h1>API  
funcionando</h1>
```

```

21          <pre>
22          <h2>Consola H2:</h2>
23          - <a href="http://
localhost:8080/h2-console">http://
localhost:8080/h2-console</a>
24
25          =====
=====
26          <h2>USUARIOS (/
usuarios)</h2>
27          =====
=====
28          - <a href="http://
localhost:8080/usuarios">http://
localhost:8080/usuarios</a>
29
30          GET  /usuarios
          listar todos
31          GET  /usuarios/{id
}          buscar por ID
32          GET  /usuarios/{id}/
nombre    obtener nombre y apellido
33          GET  /usuarios/mail/{
mail}     buscar por email
34          POST /usuarios
          crear usuario
35          PUT  /usuarios/{id
}          editar usuario
36          DELETE /usuarios/{id

```

```

36 }                eliminar usuario
37
38                =====
=====
39                <h2>PRODUCTOS (/
productos)</h2>
40                =====
=====
41                - <a href="http://
localhost:8080/productos">http://
localhost:8080/productos</a>
42
43                GET  /productos
listar todos
44                GET  /productos/
eliminados    listar eliminados
45                GET  /productos/{id
}            buscar por ID
46                POST /productos
crear producto
47                PUT  /productos/{id
}            editar producto
48                DELETE /productos/{id
}            eliminar producto
49
50                =====
=====
51                <h2>PEDIDOS (/pedidos
)</h2>

```

```

52          =====
          =====
53          - <a href="http://
localhost:8080/pedidos">http://
localhost:8080/pedidos</a>
54
55          GET  /pedidos
          listar todos
56          GET  /pedidos/{id
          }      buscar por ID
57          POST /pedidos
          crear pedido
58          PUT  /pedidos/{id
          }      editar pedido
59          DELETE /pedidos/{id
          }      baja lógica
60          POST /pedidos/asignar
          crear pedido y asignarlo a
          usuario
61
62          =====
          =====
63          <h2>CATEGORIAS (/
categorias)</h2>
64          =====
          =====
65          - <a href="http://
localhost:8080/categorias">http://
localhost:8080/categorias</a>

```

```
66
67         GET  /categorias
        listar activas
68         GET  /categorias/{id
        }      buscar por ID
69         POST /categorias
        crear categoría
70         PUT  /categorias/{id
        }      editar categoría
71         DELETE /categorias/{
        id}    baja lógica
72         </pre>
73         """;
74     }
75 }
76
```



```
1 package com.ejemploSpring.Ejercicio.  
   controller;  
2  
3 import com.ejemploSpring.Ejercicio.  
   dtos.pedido.PedidoCreate;  
4 import com.ejemploSpring.Ejercicio.  
   dtos.pedido.PedidoCreateConUsuario;  
5 import com.ejemploSpring.Ejercicio.  
   dtos.pedido.PedidoDto;  
6 import com.ejemploSpring.Ejercicio.  
   dtos.pedido.PedidoEdit;  
7 import com.ejemploSpring.Ejercicio.  
   entities.Pedido;  
8 import com.ejemploSpring.Ejercicio.  
   service.PedidoService;  
9 import lombok.RequiredArgsConstructor  
   ;  
10 import org.springframework.web.bind.  
   annotation.*;  
11  
12 import java.util.List;  
13  
14 @RestController  
15 @RequestMapping("/pedidos")  
16 @RequiredArgsConstructor  
17 public class PedidoController {  
18  
19     private final PedidoService  
       pedidoService;
```

```
20
21     // Crear pedido con detalles
22     @PostMapping
23     public PedidoDto create(@
    RequestBody PedidoCreate pedidoCreate
    ) {
24         return PedidoDto.toDto(
    pedidoService.save(pedidoCreate));
25     }
26
27     // Obtener pedido por ID
28     @GetMapping("/{id}")
29     public PedidoDto findById(@
    PathVariable Long id) {
30         return PedidoDto.toDto(
    pedidoService.findById(id));
31     }
32
33     // Obtener todos los pedidos
34     @GetMapping
35     public List<PedidoDto> findAll
    () {
36         return pedidoService.findAll
    ().stream()
37             .map(PedidoDto::toDto
    )
38             .toList();
39     }
40
```

```
41      // Editar pedido (fecha, estado,
    forma de pago)
42      @PutMapping("/{id}")
43      public PedidoDto update(@
    PathVariable Long id, @RequestBody
    PedidoEdit pedidoEdit) {
44          return PedidoDto.toDto(
    pedidoService.update(id, pedidoEdit
    ));
45      }
46
47      // Soft delete
48      @DeleteMapping("/{id}")
49      public void delete(@PathVariable
    Long id) {
50          pedidoService.deleteById(id
    ); // baja lógica
51      }
52
53      @PostMapping("/asignar")
54      public PedidoDto
    crearPedidoConUsuario(
55          @RequestBody
    PedidoCreateConUsuario dto
56      ) {
57          Pedido pedido = pedidoService
    .save(dto.pedido());
58          pedidoService.asignarPedido(
    dto.usuarioId(), pedido);
```

```
59         return PedidoDto.toDto(pedido
    );
60     }
61
62 }
```

```
1 package com.ejemploSpring.Ejercicio.  
   controller;  
2  
3 import com.ejemploSpring.Ejercicio.  
   dtos.usuario.UsuarioCreate;  
4 import com.ejemploSpring.Ejercicio.  
   dtos.usuario.UsuarioDto;  
5 import com.ejemploSpring.Ejercicio.  
   dtos.usuario.UsuarioEdit;  
6 import com.ejemploSpring.Ejercicio.  
   dtos.usuario.UsuarioNombreDto;  
7 import com.ejemploSpring.Ejercicio.  
   service.UsuarioService;  
8 import jakarta.validation.Valid;  
9 import lombok.RequiredArgsConstructor  
   ;  
10 import org.springframework.http.  
   HttpStatus;  
11 import org.springframework.web.bind.  
   annotation.*;  
12  
13 import java.util.List;  
14  
15 @RestController  
16 @RequestMapping("/usuarios")  
17 @RequiredArgsConstructor  
18 public class UsuarioController {  
19  
20     private final UsuarioService
```

```
20  usuarioService;
21
22      // Get All
23      @GetMapping("")
24      public List<UsuarioDto>
25  getUsuarios() {
26      return usuarioService.findAll
27      ();
28
29      // Get By ID
30      @GetMapping("/{id}")
31      public UsuarioDto getUsuario(@
32  PathVariable Long id) {
33      return usuarioService.
34  findById(id);
35
36      // GET solamente nombre y
37  apellido
38      @GetMapping("/{id}/nombre")
39      public UsuarioNombreDto
40  getNombreApellido(@PathVariable Long
41  id) {
42      return usuarioService.
43  getNombreApellido(id);
44
45      // Create
```

```
41     @PostMapping("")
42     @ResponseStatus(HttpStatus.
    CREATED)
43     public UsuarioDto createUsuario(@
    Valid @RequestBody UsuarioCreate
    usuarioCreate) {
44         return usuarioService.save(
    usuarioCreate);
45     }
46
47     // Update
48     @PutMapping("/{id}")
49     public UsuarioDto updateUsuario(
50         @PathVariable Long id,
51         @Valid @RequestBody
    UsuarioEdit usuarioEdit
52     ) {
53         return usuarioService.update(
    usuarioEdit, id);
54     }
55
56     // Delete
57     @DeleteMapping("/{id}")
58     @ResponseStatus(HttpStatus.
    NO_CONTENT)
59     public void deleteUsuario(@
    PathVariable Long id) {
60         usuarioService.deleteById(id
    );
```

```
61     }
62
63     @GetMapping("/mail/{mail}")
64     public UsuarioDto getByEmail(@
PathVariable String mail) {
65         return usuarioService.
findByMail(mail);
66     }
67
68 }
69
```



```
1 package com.ejemploSpring.Ejercicio.  
   controller;  
2  
3 import com.ejemploSpring.Ejercicio.  
   dtos.producto.ProductoCreate;  
4 import com.ejemploSpring.Ejercicio.  
   dtos.producto.ProductoDto;  
5 import com.ejemploSpring.Ejercicio.  
   dtos.producto.ProductoEdit;  
6 import com.ejemploSpring.Ejercicio.  
   service.ProductoService;  
7 import jakarta.validation.Valid;  
8 import lombok.RequiredArgsConstructor  
   ;  
9 import org.springframework.http.  
   HttpStatus;  
10 import org.springframework.web.bind.  
   annotation.*;  
11  
12 import java.util.List;  
13  
14 @RestController  
15 @RequestMapping("/productos")  
16 @RequiredArgsConstructor  
17 public class ProductoController {  
18  
19     private final ProductoService  
       productoService;  
20
```

```
21      // Get All
22      @GetMapping("")
23      public List<ProductoDto>
getProductos() {
24          return productoService.
findAll();
25      }
26
27      @GetMapping("/eliminados")
28      public List<ProductoDto>
getEliminados() {
29          return productoService.
findAllEliminados();
30      }
31
32      // GetByID
33      @GetMapping("/{id}")
34      public ProductoDto getProducto(@
PathVariable Long id) {
35          return productoService.
findById(id);
36      }
37
38      // Create
39      @PostMapping("")
40      @ResponseStatus(HttpStatus.
CREATED)
41      public ProductoDto createProducto
(@Valid @RequestBody ProductoCreate
```

```
41 productoCreate) {
42     return productoService.save(
    productoCreate);
43 }
44
45 // Update
46 @PutMapping("/{id}")
47 public ProductoDto updateProducto
    (
48     @PathVariable Long id,
49     @Valid @RequestBody
    ProductoEdit productoEdit
50 ) {
51     return productoService.update
    (productoEdit, id);
52 }
53
54 // Delete
55 @DeleteMapping("/{id}")
56 @ResponseStatus(HttpStatus.
    NO_CONTENT)
57 public void deleteProducto(@
    PathVariable Long id) {
58     productoService.deleteById(id
    );
59 }
60 }
```

```
1 package com.ejemploSpring.Ejercicio.  
   controller;  
2  
3  
4 import com.ejemploSpring.Ejercicio.  
   dtos.categoria.CategoriaCreate;  
5 import com.ejemploSpring.Ejercicio.  
   dtos.categoria.CategoriaDto;  
6 import com.ejemploSpring.Ejercicio.  
   dtos.categoria.CategoriaEdit;  
7 import com.ejemploSpring.Ejercicio.  
   service.CategoriaService;  
8 import lombok.RequiredArgsConstructor  
   ;  
9 import org.springframework.web.bind.  
   annotation.*;  
10  
11 import java.util.List;  
12  
13 @RestController  
14 @RequestMapping("/categorias")  
15 @RequiredArgsConstructor  
16 public class CategoriaController {  
17  
18     private final CategoriaService  
   categoriaService;  
19  
20     // Crear categoría  
21     @PostMapping
```

```
22      public CategoriaDto create(@
    RequestBody CategoriaCreate
    categoriaCreate) {
23          return categoriaService.save(
    categoriaCreate);
24      }
25
26      // Obtener categoría por ID (solo
activas)
27      @GetMapping("/{id}")
28      public CategoriaDto findById(@
    PathVariable Long id) {
29          return categoriaService.
    findById(id);
30      }
31
32      // Obtener todas las categorías
activas
33      @GetMapping
34      public List<CategoriaDto> findAll
    () {
35          return categoriaService.
    findAll();
36      }
37
38      // Editar categoría
39      @PutMapping("/{id}")
40      public CategoriaDto update(@
    PathVariable Long id,
```

```
41                                     @
    RequestBody CategoriaEdit
    categoriaEdit) {
42        return categoriaService.
    update(categoriaEdit, id);
43    }
44
45    // Soft delete
46    @DeleteMapping("/{id}")
47    public void delete(@PathVariable
    Long id) {
48        categoriaService.deleteById(
    id);
49    }
50
51 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  controller.advice;  
2  
3 import org.springframework.http.  
  HttpStatus;  
4 import org.springframework.http.  
  ResponseEntity;  
5 import org.springframework.web.bind.  
  MethodArgumentNotValidException;  
6 import org.springframework.web.bind.  
  annotation.ExceptionHandler;  
7 import org.springframework.web.bind.  
  annotation.RestControllerAdvice;  
8  
9 import java.util.HashMap;  
10 import java.util.Map;  
11  
12 @RestControllerAdvice  
13 public class AdviceController {  
14  
15  
16     // 1) Validaciones @Valid  
17  
18     @ExceptionHandler(  
        MethodArgumentNotValidException.class  
    )  
19     public ResponseEntity<Map<String  
        , String>> handleValidationErrors(  
        MethodArgumentNotValidException ex) {
```

```
20         Map<String, String> errors =
    new HashMap<>();
21
22         ex.getBindingResult().
    getFieldErrors().forEach(error ->
23             errors.put(error.
    getField(), error.getDefaultMessage
    ()))
24     );
25
26     return ResponseEntity.status(
    HttpStatus.BAD_REQUEST).body(errors);
27 }
28
29
30     // 2) No encontrado (
NullPointerException)
31
32     @ExceptionHandler(
    NullPointerException.class)
33     public ResponseEntity<Map<String
    , String>> handleNotFound(
    NullPointerException ex) {
34         Map<String, String> error =
    new HashMap<>();
35         error.put("error", ex.
    getMessage());
36         return ResponseEntity.status(
    HttpStatus.NOT_FOUND).body(error);
```



```
37     }
38
39
40     // 3) IllegalArgumentException
41
42     @ExceptionHandler(
43         IllegalArgumentException.class)
44     public ResponseEntity<Map<String
45         , String>> handleIllegalArgumentException(
46         IllegalArgumentException ex) {
47         Map<String, String> error =
48         new HashMap<>();
49         error.put("error", ex.
50         getMessage());
51         return ResponseEntity.status(
52         HttpStatus.BAD_REQUEST).body(error);
53     }
54
55     // 4) RuntimeException genérica
56
57     @ExceptionHandler(
58         RuntimeException.class)
59     public ResponseEntity<Map<String
60         , String>> handleRuntime(
61         RuntimeException ex) {
62         Map<String, String> error =
63         new HashMap<>();
64         error.put("error", ex.
```

```
55 getMessage());  
56         return ResponseEntity.status(  
    HttpStatus.INTERNAL_SERVER_ERROR).  
    body(error);  
57     }  
58 }
```

```
1 package com.ejemploSpring.Ejercicio.  
  repository;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Pedido;  
4 import org.springframework.data.jpa.  
  repository.JpaRepository;  
5 import org.springframework.stereotype  
  .Repository;  
6  
7 @Repository  
8 public interface PedidoRepository  
  extends JpaRepository<Pedido, Long> {  
9 }  
10
```

```
1 package com.ejemploSpring.Ejercicio.  
  repository;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Usuario;  
4 import org.springframework.data.jpa.  
  repository.JpaRepository;  
5 import org.springframework.stereotype  
  .Repository;  
6  
7 import java.util.Optional;  
8  
9 @Repository  
10 public interface UsuarioRepository  
  extends JpaRepository<Usuario, Long  
  > {  
11     Optional<Usuario> findByMail(  
  String email);  
12 }  
13
```

```
1 package com.ejemploSpring.Ejercicio.  
  repository;  
2  
3 import com.ejemploSpring.Ejercicio.  
  entities.Producto;  
4 import org.springframework.data.jpa.  
  repository.JpaRepository;  
5 import org.springframework.stereotype  
  .Repository;  
6  
7 import java.util.List;  
8  
9  
10 @Repository  
11 public interface ProductoRepository  
  extends JpaRepository<Producto, Long  
  > {  
12     List<Producto>  
  findAllByEliminadoFalse();  
13     List<Producto>  
  findAllByEliminadoTrue();  
14  
15 }  
16
```

```
1 package com.ejemploSpring.Ejercicio.  
    repository;  
2 import org.springframework.data.jpa.  
    repository.JpaRepository;  
3 import org.springframework.stereotype  
    .Repository;  
4  
5  
6 import com.ejemploSpring.Ejercicio.  
    entities.Categoria;  
7 import org.springframework.stereotype  
    .Repository;  
8  
9 import java.util.List;  
10  
11 @Repository  
12 public interface CategoriaRepository  
    extends JpaRepository<Categoria, Long  
        >{  
13     List<Categoria>  
        findAllByEliminadoFalse();  
14  
15 }  
16
```

```
1 package com.ejemploSpring.Ejercicio.  
   repository;  
2  
3 import com.ejemploSpring.Ejercicio.  
   entities.DetallePedido;  
4 import org.springframework.data.jpa.  
   repository.JpaRepository;  
5 import org.springframework.stereotype  
   .Repository;  
6  
7 @Repository  
8 public interface  
   DetallePedidoRepository extends  
   JpaRepository<DetallePedido, Long> {  
9 }  
10
```